

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ	8
ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	9
ВВЕДЕНИЕ	10
Актуальность темы исследования	10
Исследованность темы	11
Решаемая проблема	11
Цель работы	12
Задачи работы	12
Практическая значимость работы	13
1 Постановка задачи	14
1.1 Содержательная постановка задачи	14
1.2 Объект и предмет исследования	15
1.3 Формальная постановка задачи управления	15
1.4 Требования к системе	16
1.4.1 Функциональные требования	16
1.4.2 Нефункциональные требования	17
1.4.3 Ограничения	17
1.5 Критерии оценки качества решения	17
2 Обзор существующих решений	19
2.1 Подходы к локальному планированию траектории	19
2.1.1 Реактивные методы	19
2.1.2 Классическое модельное прогнозирующее управление (MPC)	19
2.1.3 Стохастические методы: MPPI	20
2.2 Методы SLAM для замкнутых помещений	21
2.3 Навигационные стеки	21

2.4	Сравнительный анализ и обоснование выбора.....	22
3	Исследование и построение решения задачи.....	23
3.1	Кинематическая схема омни-платформы.....	23
3.2	Кинематика в непрерывном времени.....	25
3.3	Связь с угловыми скоростями колёс	26
3.4	Дискретная модель	27
3.5	Ограничения.....	27
3.6	Принцип модельного прогнозирующего управления.....	27
3.7	Стохастическая реализация MPPI	28
3.8	Архитектура функции потерь.....	29
3.9	Направленный штраф поля препятствий.....	30
3.10	Интерпретация весовых коэффициентов	31
3.11	Декомпозиция на критики Nav2/MPPI.....	32
3.12	Контур восприятия цели	32
4	Описание практической части.....	34
4.1	Выбор средств разработки	34
4.2	Состав узлов и их назначение	35
4.3	Модель робота	35
4.4	Среда симуляции и подвижные сущности	36
4.5	Контур восприятия цели	37
4.6	Формирование навигационных целей.....	38
4.7	Локальное управление с MPPI.....	39
4.7.1	Пользовательский критик <code>TargetCameraCritic</code>	40
4.7.2	Пользовательский критик <code>PotentialFieldCritic</code>	41
4.8	SLAM.....	42
4.9	Пример рабочей сцены	42
5	Экспериментальная часть	44
5.1	Методика проведения серии запусков	44

5.2	Метрики	44
5.2.1	Распределение режимов сопровождения	44
5.2.2	Видимость цели	45
5.2.3	Метрики безопасности	45
5.2.4	Плавность управления	45
5.3	Результаты одного запуска	46
5.4	Агрегированные результаты по серии экспериментов	48
5.5	Обсуждение результатов	50
ЗАКЛЮЧЕНИЕ		52
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ		55
СПИСОК ИЛЛЮСТРАТИВНОГО МАТЕРИАЛА		56

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

MPC	Model Predictive Control, модельное прогнозирующее управление.
MPPI	Model Predictive Path Integral, стохастическая реализация MPC.
ROS	Robot Operating System, фреймворк программных модулей для робототехники.
SDF	Simulation Description Format, формат описания моделей среды Gazebo.
URDF	Unified Robot Description Format, формат описания модели робота.
SLAM	Simultaneous Localization and Mapping, одновременная локализация и построение карты.
TF	Transform tree, дерево преобразований координатных систем в ROS.
Nav2	Navigation 2, пакет навигационного стека ROS 2.
RGB	Red-Green-Blue, трёхканальная цветовая модель изображения.
HSV	Hue-Saturation-Value, цветовая модель оттенок-насыщенность-яркость.
FOV	Field of View, угол поля зрения камеры.
VNC	Virtual Network Computing, протокол удалённого доступа к графическому интерфейсу.
RMS	Root Mean Square, среднеквадратичное значение.

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Сопровождение цели	Режим работы робота, в котором робот автономно удерживает заданную дистанцию и ориентацию относительно подвижного пользователя.
Функция потерь	Скалярный функционал, подлежащий минимизации оптимизатором на горизонте прогнозирования и формализующий требования задачи управления.
Критик траектории	Независимый программный модуль, вычисляющий отдельное слагаемое функции потерь по заданной сэмплированной траектории в рамках архитектуры МРРІ.
Горизонт прогнозирования	Число шагов вперёд, на которое модель предсказывает состояние системы при решении задачи оптимизации управления.
Costmap	Двумерная карта стоимости перемещения, формируемая на основе карты занятости и расширенная штрафом за приближение к препятствиям.

ВВЕДЕНИЕ

Актуальность темы исследования

В последние годы сервисные мобильные роботы быстро проникают в бытовую сферу: от роботов-пылесосов и автоматических курьеров до персональных ассистентов, сопровождающих пользователя в домашних условиях. Ключевой сценарий применения таких систем — следование за человеком в ограниченной среде, где одновременно должны обеспечиваться безопасность движения, устойчивость поведения при временной потере наблюдаемости цели и учёт динамически меняющейся обстановки (перемещаемые предметы быта, домашние животные, другие люди в квартире).

Классические подходы к управлению мобильными роботами, основанные на раздельных контурах глобального и локального планирования и простых реактивных алгоритмах типа Dynamic Window Approach, плохо справляются с одновременной формализацией нескольких разнородных требований — удержание дистанции и центрирование цели, избежание столкновений, сохранение плавности управления. Современное развитие вычислительной техники и стохастических методов оптимизации сделало практически применимыми методы модельного прогнозирующего управления (MPC), в том числе их стохастическую выборка-ориентированную реализацию MPPi, поддерживающую нелинейные модели и негладкие составные функции потерь.

Применение MPPi в задаче сопровождения пользователя в квартире непосредственно отражает современные тренды и является актуальной инженерной и научно-прикладной задачей.

Исследованность темы

Кинематические и динамические модели омни-роботов с колёсами Mecanum детально исследованы в работах по мобильной робототехнике; в литературе известны соотношения, связывающие скорости вращения колёс с обобщёнными скоростями корпуса и учитывающие ограничения исполнительных механизмов [1]. Применение MPC для Mecanum-роботов в задачах обхода препятствий рассматривалось в ряде работ, демонстрирующих преимущества оптимизационного подхода по сравнению с классическими реактивными регуляторами [1, 2].

Стохастическая реализация MPC — MPPI — получила практическое применение в навигационном стеке ROS 2 (Nav2), где имеется полноценная реализация соответствующего локального планировщика. Параллельно широко развиваются алгоритмы SLAM (Gmapping, Hector SLAM, Cartographer, slam_toolbox), позволяющие одновременно решать задачу построения карты и локализации робота в помещении [3, 4].

Вместе с тем целостное решение, объединяющее в одной воспроизводимой среде задачу сопровождения пользователя (трекинг цели по камере + позиционирование в карте), MPPI-контроллер с критиками, настроенными под прикладные требования (дистанция до цели, центрирование, обход динамических препятствий, устойчивое поведение при потере цели), в литературе практически не описано в открытом виде. Данная работа отвечает на этот запрос.

Решаемая проблема

В работе решается проблема разработки программно-алгоритмического комплекса автономного мобильного омни-робота, который в условиях типовой квартирной среды с подвижными препятствиями способен

удерживать подвижного пользователя в рабочей зоне камеры и поддерживать заданную дистанцию, избегая столкновений и сохраняя устойчивое поведение при временной потере цели.

Цель работы

Целью ВКР является разработка и экспериментальное исследование программного обеспечения автономного мобильного омни-робота, обеспечивающего сопровождение пользователя в условиях квартирной среды с динамическими препятствиями на базе стохастической реализации модельного прогнозирующего управления.

Задачи работы

Для достижения поставленной цели сформулированы следующие задачи:

1. Проанализировать существующие подходы к локальному планированию траектории мобильных роботов в ограниченной среде, в том числе методы модельного прогнозирующего управления и их стохастические реализации.
2. Сформировать математическую модель омни-платформы с колёсами Mecanum, включая кинематические соотношения и ограничения управления.
3. Формализовать составную функцию потерь, отражающую прикладные требования.
4. Разработать воспроизводимую программно-симуляционную среду на базе Docker, ROS 2 и Gazebo для отладки и валидации алгоритмов управления.
5. Реализовать модель омни-робота с 2D-лидаром и фронтальной RGB-

камерой, создать узлы детекции цели и оценки ее положения.

6. Реализовать критик МРРІ-контроллера, обеспечивающий удержание цели в центре кадра камеры, и интегрировать его в навигационный стек Nav2.
7. Реализовать критик МРРІ-контроллера, формирующий направленный (анизотропный) штраф на основе локального поля препятствий, для повышения безопасности голономной омни-платформы при сближении с препятствиями.
8. Автоматизировать процесс сбора статистики и визуализации результатов.

Практическая значимость работы

Результаты работы могут быть использованы:

- как основа прототипа персонального сервисного робота, выполняющего автономное сопровождение пользователя в бытовой среде;
- как воспроизводимый программно-симуляционный стенд для исследования и настройки алгоритмов локального планирования и трекинга в условиях квартиры;
- для сравнения различных конфигураций критиков МРРІ и подтверждения того, что декомпозиция составной функции потерь на независимые критики является практичным способом формализации разнородных требований к поведению робота.

1 ПОСТАНОВКА ЗАДАЧИ

В настоящем разделе формулируется содержательная и формальная постановка задачи, определяются объект и предмет исследования, описываются требования к разрабатываемой системе и ограничения, накладываемые на решение.

1.1 Содержательная постановка задачи

Требуется разработать программное обеспечение мобильного робота на омни-платформе с колёсами типа Mecanum, оснащённого фронтальной RGB-камерой и двумерным лазерным дальномером (2D-лидаром), которое в условиях типовой квартирной среды с подвижными препятствиями обеспечивает:

1. автономное обнаружение подвижного пользователя в поле зрения камеры;
2. сопровождение пользователя с удержанием целевой дистанции до него в заданных пределах;
3. удержание пользователя в центральной зоне поля зрения камеры по горизонтальной оси;
4. избегание столкновений со статическими и подвижными препятствиями;
5. безопасное поведение при временной потере наблюдаемости цели (остановка с последующим поиском).

Среда эксплуатации — квартира с характерными габаритами и типичным набором бытовых препятствий. Среда считается частично известной: карта помещения формируется в реальном времени средствами SLAM по данным лидара.

1.2 Объект и предмет исследования

Объект исследования: автономный мобильный колёсный робот на омни-платформе, предназначенный для задачи сопровождения пользователя в ограниченной помещении.

Предмет исследования: алгоритмы и программные средства локального планирования траектории и управления движением такого робота в условиях динамически меняющейся среды.

1.3 Формальная постановка задачи управления

Вектор состояния робота в глобальной системе координат определяется как

$$\mathbf{x}_k = \begin{bmatrix} x_k & y_k & \psi_k \end{bmatrix}^\top, \quad (1.1)$$

где x_k, y_k — координаты центра масс робота в глобальной системе, ψ_k — угол ориентации корпуса относительно оси x глобальной системы координат.

Вектор управления, доступный для омни-платформы, имеет вид

$$\mathbf{u}_k = \begin{bmatrix} v_{x,k} & v_{y,k} & \omega_k \end{bmatrix}^\top, \quad (1.2)$$

где $v_{x,k}, v_{y,k}$ — составляющие линейной скорости корпуса в связанной с роботом системе координат x_{BUB} , ω_k — угловая скорость вращения корпуса относительно вертикальной оси.

Положение подвижного пользователя (цели) в глобальной системе координат описывается вектором

$$\mathbf{x}_k^t = \begin{bmatrix} x_k^t & y_k^t \end{bmatrix}^\top. \quad (1.3)$$

Пусть $d_k = \|\mathbf{p}_k - \mathbf{x}_k^t\|_2$ — расстояние от робота до цели, где $\mathbf{p}_k = [x_k, y_k]^\top$. Пусть ϕ_k — угол направления на цель относительно оси камеры. Тогда задача управления формулируется как поиск последовательности управлений $\{\mathbf{u}_k\}$,

минимизирующей составную функцию потерь

$$J = \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k, \mathbf{x}_k^t) + \ell_f(\mathbf{x}_N), \quad (1.4)$$

при ограничениях модели движения $\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$, ограничениях на управление $\mathbf{u}_k \in \mathcal{U}$ и условиях безопасности

$$\|\mathbf{p}_k - \mathbf{p}_{i,k}^{\text{obs}}\|_2 \geq d_{\min}, \quad \forall i, \forall k, \quad (1.5)$$

где $\mathbf{p}_{i,k}^{\text{obs}}$ — положение i -го препятствия на шаге k , $d_{\min}$ — минимально допустимое расстояние.

Конкретный вид функции $\ell(\cdot)$, отражающий прикладные требования, и детализация модели $f(\cdot)$ приведены в главе 3.

1.4 Требования к системе

Разрабатываемая система должна удовлетворять следующим требованиям:

1.4.1 Функциональные требования

1. Автономное обнаружение цели в кадре камеры по характерному визуальному признаку (в рамках модельной реализации — по цветовой сегментации).
2. Пересчёт положения обнаруженной цели в карту помещения с использованием данных лидара, дерева преобразований координат и координат цели в кадре камеры.
3. Удержание целевой дистанции d^* до пользователя в пределах заданной погрешности.
4. Удержание цели в центральной зоне кадра камеры по горизонтальной оси.
5. Обход статических и динамических препятствий с соблюдением

минимальной дистанции $d_{\min}$.

6. Поиск цели в случае ее потери.

1.4.2 Нефункциональные требования

1. Воспроизводимость среды запуска за счёт контейнеризации.
2. Возможность запуска всей симуляционной цепочки одной командой.
3. Модульность архитектуры: перенос алгоритмических модулей на реальное оборудование без изменения их внутренней логики.
4. Параметризуемость поведения системы через конфигурационные файлы без необходимости перекомпиляции.
5. Удалённый доступ к графическому интерфейсу симулятора независимо от особенностей графической подсистемы хоста.

1.4.3 Ограничения

1. Работа выполняется в рамках физически-реалистичной симуляции Gazebo, интеграция на реальное оборудование выходит за рамки данной ВКР.
2. Пользователь представлен моделью подвижного цветного объекта (синий цилиндр), что позволяет сконцентрироваться на контуре управления; замена детектора цели на промышленное решение компьютерного зрения технически осуществима, но не включена в объём работы.
3. Сенсорный набор ограничен 2D-лидаром и фронтальной RGB-камерой.

1.5 Критерии оценки качества решения

Качество разработанного решения оценивается следующими количественными характеристиками, вычисляемыми по записанным `rosbag2` (подробное описание методики и формальные определения — в главе 5):

- доля времени каждого режима сопровождения $\bar{\tau}_m$ — доминирование

режима Сопровождение цели подтверждает работоспособность связки восприятия и контроллера;

- доля времени видимости цели $\bar{\mathbf{v}} \in [0,1]$ — интегральная характеристика устойчивости трекера и поведения при загораживаниях;
- число фактических контактов с препятствиями N_c и число заходов в зону риска N_r , а также доля предотвращённых контактов $\rho = N_p/N_r$ — характеристики безопасности относительно ограничения (1.5);
- среднеквадратичная норма приращения управления $\text{RMS}(\|\Delta \mathbf{u}_k\|_2)$ — характеристика плавности управления.

2 ОБЗОР СУЩЕСТВУЮЩИХ РЕШЕНИЙ

В настоящем разделе рассмотрены существующие подходы к решению поставленной задачи. Обзор структурирован по трём независимым составляющим: методы локального планирования движения, методы построения карты и локализации (SLAM) и программные навигационные стеки, на базе которых практически реализуются такие решения. В конце раздела выполнено обоснование выбранного подхода.

2.1 Подходы к локальному планированию траектории

2.1.1 Реактивные методы

Исторически первыми практическими подходами были реактивные локальные планировщики: Dynamic Window Approach (DWA), Timed Elastic Band (TEB), методы на основе потенциальных полей. DWA выбирает команду скорости из допустимого по динамике окна, максимизирующую заданную эвристику. Такие подходы обладают высокой вычислительной эффективностью, однако с ростом числа разнородных требований эвристика становится громоздкой и плохо поддаётся настройке, что ограничивает их применимость в задаче сопровождения.

2.1.2 Классическое модельное прогнозирующее управление (MPC)

Модельное прогнозирующее управление [2] представляет собой класс методов оптимального управления, в котором на каждом шаге решается задача оптимизации с конечным горизонтом и использованием предсказательной модели объекта и составной функции потерь. MPC естественным образом формализует произвольное число разнородных требований в виде слагаемых функционала и ограничений, а также легко учитывает физические ограничения на управление.

Применение классического нелинейного MPC (например, с решателями на основе SQP [2]) требует аналитической гладкости модели и штрафов, а также значительных вычислительных ресурсов для поиска глобального минимума. Для роботов с колёсами Mesarum подходы MPC рассмотрены в работе [1], где учитываются как кинематика, так и электрическая динамика приводов, но вычислительная сложность остаётся высокой.

2.1.3 Стохастические методы: MPPI

Model Predictive Path Integral (MPPI) — выборка-ориентированная стохастическая реализация MPC. В каждом такте MPPI формирует набор сэмплов управления путём добавления гауссовского шума к номинальному управлению, прогнозирует траектории с использованием модели системы, вычисляет стоимость каждой траектории и формирует обновление номинального управления как экспоненциально взвешенное среднее по сэмплам.

Преимущества MPPI перед классическим MPC в контексте задачи сопровождения пользователя:

- отсутствие требования гладкости функции потерь и модели системы, что позволяет использовать штрафы по данным costmap и произвольные логические условия (например, индикатор видимости цели);
- естественная параллелизация по независимым траекториям, позволяющая эффективно использовать современные CPU/GPU;
- устойчивость к локальным минимумам за счёт случайного сэмплирования.

Недостатки MPPI: необходимость тонкой настройки параметров сэмплирования (число сэмплов, дисперсия шума, температура) и чувствительность качества решения к количеству сэмплов.

2.2 Методы SLAM для замкнутых помещений

В закрытых помещениях, где отсутствует надёжный сигнал глобального позиционирования, навигация строится на методах SLAM. Для 2D-лидара наиболее распространены:

- Gmapping — частично-фильтровое решение Rao-Blackwellized particle filter SLAM;
- Hector SLAM — сопоставление сканов на основе градиентной оптимизации, не требующее одометрии;
- Cartographer — решение Google с графовой оптимизацией и замыканием цикла;
- slam_toolbox — прямой аналог Cartographer в экосистеме ROS 2 с поддержкой асинхронного online-режима.

Сравнительные исследования [3, 4] показывают, что для задач квартирной навигации с 2D-лидаром решения класса slam_toolbox и Cartographer обеспечивают лучший баланс точности карты, стабильности локализации и вычислительной нагрузки. Hector SLAM выигрывает в простоте запуска и отсутствии требования одометрии, но проигрывает в точности на длинных траекториях из-за отсутствия замыкания циклов.

2.3 Навигационные стеки

Практически значимые решения строятся не с нуля, а на основе готовых навигационных стеков, обеспечивающих интеграцию планировщиков, контроллеров, поведенческих деревьев и средств преобразования систем координат.

В среде ROS 2 де-факто стандартом является стек Navigation 2 (Nav2): он включает сервер глобального планировщика, сервер локального контроллера с набором плагинов, сервер поведений (spin, wait, backup)

и BT Navigator, выполняющий поведенческое дерево. Среди локальных планировщиков Nav2 имеются реализации DWB, TEB, Regulated Pure Pursuit и MPPI (`nav2_mppi_controller`), что делает Nav2 удобной базой для задачи настоящей работы.

2.4 Сравнительный анализ и обоснование выбора

Сопоставление рассмотренных подходов с требованиями главы 1 приводит к следующим выводам.

1. Реактивные методы (DWA, потенциальные поля) не позволяют системно формализовать одновременно требования по удержанию дистанции и центрированию цели, поэтому не выбираются.
2. Классический нелинейный MPC пригоден для формулировки задачи, но требует гладких штрафов, что затрудняет использование готовых инструментов Nav2 и costmap.
3. MPPI поддерживает произвольные негладкие штрафы, естественным образом реализуется через архитектуру критиков, легко интегрируется в Nav2, имеет открытую реализацию для омни-платформ.
4. Для построения карты и локализации выбрано решение `slam_toolbox` как современный, поддерживаемый аналог Cartographer в экосистеме ROS 2.
5. Для интеграции компонент выбран стек Nav2, что упрощает построение поведенческого слоя и использует готовую реализацию MPPI.

Ни одно из существующих готовых решений не покрывает полностью требования задачи в части одновременного удержания дистанции, центрирования цели и обхода подвижных препятствий с устойчивым поведением при потере цели, поэтому в рамках ВКР в состав MPPI-контроллера добавлены пользовательские критики, формализующие соответствующие требования (подробно описано в главе 4).

3 ИССЛЕДОВАНИЕ И ПОСТРОЕНИЕ РЕШЕНИЯ ЗАДАЧИ

Построена математическая модель омни-платформы, формализован принцип МРРІ-управления и сформирована составная функция потерь, отражающая прикладные требования из главы 1.

3.1 Кинематическая схема омни-платформы

Робот представляет собой платформу с четырьмя колёсами типа Mecanum, расположенными по углам прямоугольника со сторонами $2L$ (вдоль оси x_B) и $2l$ (вдоль оси y_B) (рисунок 3.1). Ориентация роликов колёс составляет угол $\pm 45^\circ$ к продольной оси платформы и зеркально противоположна у соседних колёс.

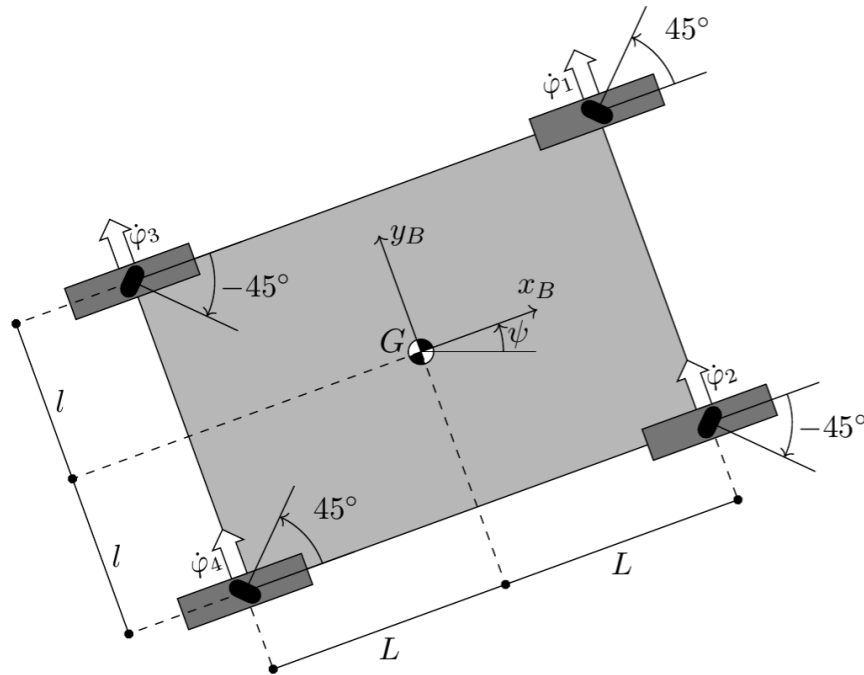


Рисунок 3.1 – Кинематическая схема омни-платформы с колёсами Mecanum

Обозначения, используемые далее (в соответствии с рисунком 3.1):

- G — геометрический центр платформы, совпадающий с центром связанной системы координат $x_B y_B$;

- ψ — угол ориентации корпуса относительно оси x глобальной системы координат;
- L — половина колёсной базы (расстояние от центра до оси колеса по оси x_B);
- l — половина колёсной колеи (расстояние от центра до оси колеса по оси y_B);
- $\dot{\phi}_i, i = 1, \dots, 4$ — угловые скорости соответствующих колёс;
- r — радиус колеса.

Принцип работы колеса Mecanum показан на рисунке 3.2: за счёт расположения свободно вращающихся роликов под углом 45° к оси вращения колеса сила реакции опоры создаёт составляющую как в продольном, так и в поперечном направлении, что позволяет платформе с четырьмя такими колёсами совершать голономное перемещение в плоскости [1].

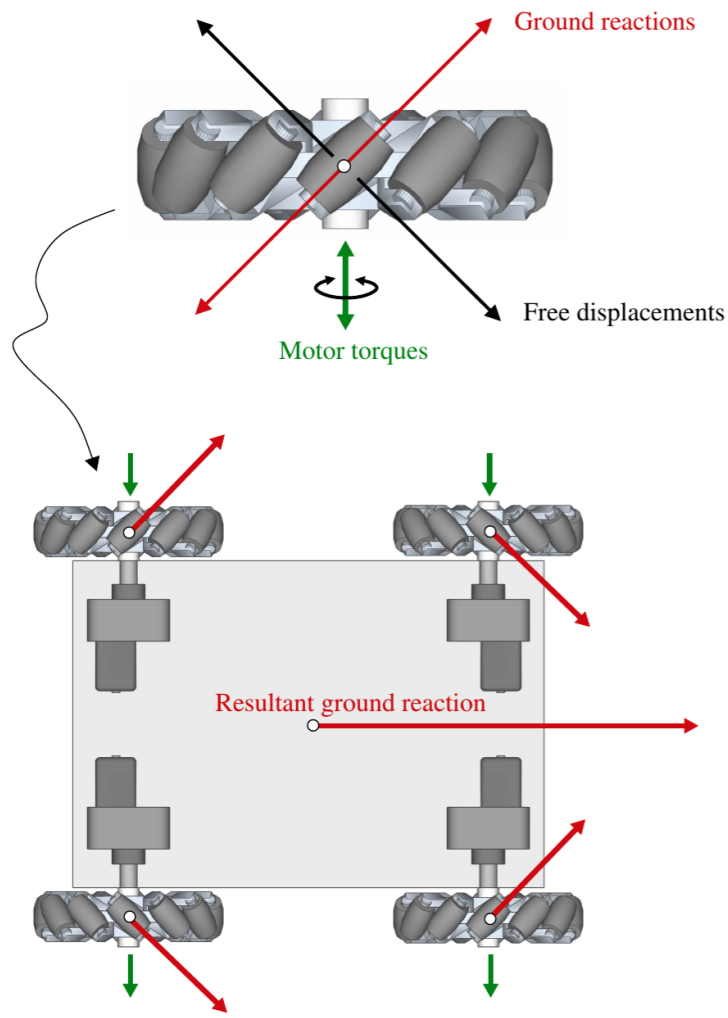


Рисунок 3.2 – Принцип действия колеса Mecanum и формирование результирующей реакции опоры

3.2 Кинематика в непрерывном времени

Пусть v_x , v_y — продольная и поперечная линейные скорости корпуса в связанной системе координат, $\omega = \dot{\psi}$ — угловая скорость. Полный вектор обобщённых скоростей корпуса имеет вид $\mathbf{u} = \begin{bmatrix} v_x & v_y & \omega \end{bmatrix}^T$.

Связь между скоростью в связанной системе и производной глобальных координат задаётся поворотом на угол ψ :

$$\dot{x} = v_x \cos \psi - v_y \sin \psi, \quad (3.1)$$

$$\dot{y} = v_x \sin \psi + v_y \cos \psi, \quad (3.2)$$

$$\dot{\psi} = \omega. \quad (3.3)$$

В матричной форме

$$\dot{\mathbf{x}} = R(\psi) \mathbf{u}, \quad R(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (3.4)$$

3.3 Связь с угловыми скоростями колёс

Для четырёхколёсной омни-платформы с радиусом колеса r , полубазой L и полуколеёй l прямая кинематика связывает обобщённые скорости корпуса $\mathbf{u}$ с угловыми скоростями колёс $\dot{\phi}_i$ следующим образом [1]:

$$\begin{bmatrix} \dot{\phi}_1 \\ \dot{\phi}_2 \\ \dot{\phi}_3 \\ \dot{\phi}_4 \end{bmatrix} = \frac{1}{r} \begin{bmatrix} 1 & -1 & -(L+l) \\ 1 & 1 & (L+l) \\ 1 & 1 & -(L+l) \\ 1 & -1 & (L+l) \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix}, \quad (3.5)$$

где строки матрицы соответствуют колёсам 1–4 в нумерации рисунка 3.1 с учётом знаков ориентации роликов ($+45^\circ$ для колёс 1 и 4, -45° для колёс 2 и 3).

Обратное соотношение, используемое для проверки достижимости управления, записывается как

$$\begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix} = \frac{r}{4} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 \\ -\frac{1}{L+l} & \frac{1}{L+l} & -\frac{1}{L+l} & \frac{1}{L+l} \end{bmatrix} \begin{bmatrix} \dot{\phi}_1 \\ \dot{\phi}_2 \\ \dot{\phi}_3 \\ \dot{\phi}_4 \end{bmatrix}. \quad (3.6)$$

3.4 Дискретная модель

Для цифровой реализации контроллера непрерывная модель дискретизируется с шагом Δt методом Эйлера:

$$x_{k+1} = x_k + \Delta t (v_{x,k} \cos \psi_k - v_{y,k} \sin \psi_k), \quad (3.7)$$

$$y_{k+1} = y_k + \Delta t (v_{x,k} \sin \psi_k + v_{y,k} \cos \psi_k), \quad (3.8)$$

$$\psi_{k+1} = \psi_k + \Delta t \omega_k. \quad (3.9)$$

В компактной форме

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k). \quad (3.10)$$

3.5 Ограничения

Физические ограничения на скорости и ускорения корпуса имеют вид

$$|v_{x,k}| \leq v_x^{\max}, \quad |v_{y,k}| \leq v_y^{\max}, \quad |\omega_k| \leq \omega^{\max}, \quad (3.11)$$

$$|\dot{v}_{x,k}| \leq a_x^{\max}, \quad |\dot{v}_{y,k}| \leq a_y^{\max}, \quad |\dot{\omega}_k| \leq \alpha^{\max}. \quad (3.12)$$

Условие безопасности относительно препятствий записывается как

$$d_{i,k} = \|\mathbf{p}_k - \mathbf{p}_{i,k}^{\text{obs}}\|_2 \geq d_{\min}, \quad i = 1, \dots, n_{\text{obs}}. \quad (3.13)$$

3.6 Принцип модельного прогнозирующего управления

На шаге k модельное прогнозирующее управление (МРС) формирует последовательность управляющих воздействий

$$\mathbf{U}_k^* = \left\{ \mathbf{u}_{k|k}^*, \mathbf{u}_{k+1|k}^*, \dots, \mathbf{u}_{k+N-1|k}^* \right\}, \quad (3.14)$$

где N — горизонт прогнозирования. Прогноз состояния строится рекурсивно с использованием модели (3.10):

$$\mathbf{x}_{k+j+1|k} = f(\mathbf{x}_{k+j|k}, \mathbf{u}_{k+j|k}), \quad j = 0, \dots, N-1. \quad (3.15)$$

Оптимизируемый функционал представляет сумму этапных и терминальных затрат:

$$J_k = \sum_{j=0}^{N-1} \ell(\mathbf{x}_{k+j|k}, \mathbf{u}_{k+j|k}) + \ell_f(\mathbf{x}_{k+N|k}). \quad (3.16)$$

По принципу receding horizon применяется только первое управление $\mathbf{u}_k = \mathbf{u}_{k|k}^*$, после чего горизонт сдвигается, измеряется новое состояние и задача оптимизации решается заново.

3.7 Стохастическая реализация MPPI

В работе используется стохастическая реализация MPC — MPPI, имеющаяся в составе пакета `nav2_mppi_controller` навигационного стека Nav2 [2]. Для каждого сэмпла $m = 1, \dots, M$ генерируется возмущение управления

$$\boldsymbol{\varepsilon}_j^{(m)} \sim \mathcal{N}(0, \Sigma), \quad m = 1, \dots, M, \quad j = 0, \dots, N-1, \quad (3.17)$$

и формируется траектория m -го сэмпла с управлением

$$\mathbf{u}_{k+j}^{(m)} = \bar{\mathbf{u}}_{k+j} + \boldsymbol{\varepsilon}_j^{(m)}, \quad (3.18)$$

где $\bar{\mathbf{u}}_{k+j}$ — номинальное управление на шаге j .

Суммарная стоимость m -й траектории

$$S^{(m)} = \sum_{j=0}^{N-1} \ell(\mathbf{x}_{k+j}^{(m)}, \mathbf{u}_{k+j}^{(m)}) + \ell_f(\mathbf{x}_{k+N}^{(m)}). \quad (3.19)$$

Вес траектории формируется экспоненциальным правилом

$$w^{(m)} = \frac{\exp\left(-\frac{1}{\lambda} S^{(m)}\right)}{\sum_{q=1}^M \exp\left(-\frac{1}{\lambda} S^{(q)}\right)}, \quad (3.20)$$

где $\lambda > 0$ — температурный параметр (в реализации Nav2 MPPI обозначается `temperature`).

Обновление номинального управления происходит по взвешенному среднему возмущений:

$$\bar{\mathbf{u}}_{k+j}^{\text{new}} = \bar{\mathbf{u}}_{k+j} + \sum_{m=1}^M w^{(m)} \boldsymbol{\varepsilon}_j^{(m)}. \quad (3.21)$$

3.8 Архитектура функции потерь

Согласно требованиям задачи (глава 1) функция потерь должна одновременно обеспечивать:

- поддержание целевой дистанции до пользователя;
- центрирование цели в поле зрения камеры по горизонтали;
- безопасность относительно препятствий;
- плавность управляющих воздействий;
- корректное поведение при потере наблюдаемости цели.

Введём следующие отклонения на шаге k :

$$e_k^d = \|\mathbf{p}_k - \mathbf{x}_k^t\|_2 - d^*, \quad (3.22)$$

$$e_k^\phi = \text{atan2}(y_k^t - y_k, x_k^t - x_k) - \psi_k, \quad (3.23)$$

где d^* — желаемая дистанция, а e_k^ϕ приводится к $(-\pi, \pi]$.

Этапная функция потерь принимается в виде взвешенной суммы

$$\begin{aligned}\ell_k = & w_d (e_k^d)^2 + w_\phi (e_k^\phi)^2 + w_u \|\mathbf{u}_k\|_2^2 \\ & + w_{\Delta u} \|\mathbf{u}_k - \mathbf{u}_{k-1}\|_2^2 + w_{\text{obs}} \Psi_{\text{obs},k} \\ & + w_{\text{pf}} \Psi_{\text{pf},k} + w_{\text{hold}} \Psi_{\text{hold},k},\end{aligned}\tag{3.24}$$

где

$$\Psi_{\text{obs},k} = \sum_{i=1}^{n_{\text{obs}}} \frac{1}{(d_{i,k} - d_{\text{safe}})^2 + \varepsilon}, \quad \varepsilon > 0,\tag{3.25}$$

$$\Psi_{\text{hold},k} = (1 - v_k) \|\mathbf{u}_k\|_2^2,\tag{3.26}$$

а $v_k \in \{0,1\}$ — бинарный индикатор видимости цели. Слагаемое $\Psi_{\text{pf},k}$ задаёт направленный (анизотропный) штраф вдоль градиента поля препятствий и определено в подразделе 3.9. Терминальная стоимость имеет вид

$$\ell_f = w_f^d (e_{k+N}^d)^2 + w_f^\phi (e_{k+N}^\phi)^2.\tag{3.27}$$

3.9 Направленный штраф поля препятствий

Изотропный барьер (3.25) штрафует только сам факт близости препятствия и не различает направления движения, что для голономной омни-платформы (свободный выбор v_x, v_y) недостаточно: сэмплы, идущие вдоль края препятствия, и сэмплы, направленные в препятствие, получают одинаковый штраф. Для устранения этого эффекта в функцию потерь дополнительно вводится направленный (потенциального поля) штраф $\Psi_{\text{pf},k}$.

Пусть $c(x,y) \in [0,1]$ — нормированное на максимум значение costmap в точке (x,y) , ∇c — его конечно-разностный градиент. Введём внутреннее расстояние до препятствия $\tilde{d}(c)$, восстанавливаемое из инфляционной модели costmap. Тогда

$$\hat{\mathbf{f}}_k = -\frac{\nabla c(\mathbf{p}_k)}{\|\nabla c(\mathbf{p}_k)\|_2}\tag{3.28}$$

— единичное направление отталкивания от препятствия в точке $\mathbf{p}_k$.

Слагаемое $\Psi_{\text{pf},k}$ строится как сумма двух неотрицательных составляющих: проксимальной и направленной. Проксимальная составляющая активна, когда восстановленное расстояние до препятствия меньше d_{pf}^* :

$$\rho_k = \max(0, d_{\text{pf}}^* - \tilde{d}(c(\mathbf{p}_k))). \quad (3.29)$$

Направленная составляющая штрафует движение в сторону препятствия — проекцию единичного вектора скорости $\hat{\mathbf{v}}_k = \mathbf{v}_k / \|\mathbf{v}_k\|_2$ на направление, противоположное отталкиванию:

$$\sigma_k = \max(0, -\langle \hat{\mathbf{v}}_k, \hat{\mathbf{f}}_k \rangle) (1 + \rho_k). \quad (3.30)$$

Полное слагаемое имеет вид

$$\Psi_{\text{pf},k} = \mathbf{1}[c(\mathbf{p}_k) \geq c_{\text{act}}] (w_\rho \rho_k + w_\sigma \sigma_k), \quad (3.31)$$

где $\mathbf{1}[\cdot]$ — индикатор активации по порогу c_{act} , а w_ρ, w_σ — внутренние веса проксимальной и направленной составляющих.

Активация по порогу c_{act} исключает штраф вдали от препятствий, что делает его локальным регуляризатором поведения и не искажает преследование цели в свободном пространстве. На расстоянии до текущей цели меньше d_{ng} слагаемое также обнуляется, чтобы исключить конфликт с подходом к финальной позе [2].

3.10 Интерпретация весовых коэффициентов

Веса $w_{(\cdot)}$ регулируют относительную значимость слагаемых функции потерь. Характерные соотношения:

- $w_{\text{obs}}, w_{\text{pf}} \gg w_d, w_\phi$ — приоритет безопасности над точностью сопровождения;
- увеличение $w_{\Delta u}$ уменьшает рывки управления: $\|\mathbf{u}_k - \mathbf{u}_{k-1}\|_2 \rightarrow 0$;

- $w_{\text{hold}} \gg 0$ при $\mathbf{v}_k = 0$ заставляет робот остановиться: $\mathbf{u}_k \rightarrow \mathbf{0}$;
- внутренний w_{σ} в (3.30) подавляет движение в препятствие на голономных направлениях, не мешая движению вдоль границы препятствия.

3.11 Декомпозиция на критики Nav2/MPPI

Любое слагаемое функции потерь в (3.24) представляется независимым модулем — критиком траектории. Формально

$$\ell_k = \sum_{r=1}^R \ell_k^{(r)}, \quad (3.32)$$

$$S^{(m)} = \sum_{j=0}^{N-1} \sum_{r=1}^R \ell_{k+j}^{(r,m)}, \quad (3.33)$$

где $\ell_k^{(r)}$ — вклад r -го критика. Такая декомпозиция обеспечивает:

- независимую параметрическую настройку каждого требования;
- интерпретируемость поведения в терминах отдельных целей;
- модульное расширение системы при добавлении новых требований к управлению.

Соответствие слагаемых функции (3.24) критикам реализации Nav2 MPPI приведено в главе 4.

3.12 Контур восприятия цели

Для связи функции потерь с измерениями сенсоров необходимо пересчитать положение обнаруженной цели в то же координатное пространство, в котором формируются прогнозы траектории. Схема пересчёта включает следующие этапы:

1. детектирование цели по цветовой сегментации изображения с фронтальной камеры;

2. вычисление углового направления на цель по положению центра ограничивающего прямоугольника и известному горизонтальному углу поля зрения камеры;
3. оценка дальности до цели по данным 2D-лидара в окрестности полученного направления (медиана допустимых значений в заданном окне индексов);
4. преобразование точки из связанной с роботом системы координат в глобальную карту через дерево преобразований TF.

Такая схема обеспечивает согласованность математической модели и измерений, а также позволяет использовать один и тот же кадр карты для всех слагаемых функции потерь.

4 ОПИСАНИЕ ПРАКТИЧЕСКОЙ ЧАСТИ

Раздел описывает практическую реализацию программного комплекса: средства разработки, архитектуру, ключевые модули и их интерфейсы.

4.1 Выбор средств разработки

В качестве среды разработки выбраны следующие программные компоненты, обеспечивающие необходимую воспроизводимость и модульность системы.

- Robot Operating System 2 (ROS 2), дистрибутив Humble Hawksbill. Фреймворк обеспечивает стандартизованную модель распределённого программного обеспечения робототехнической системы: узлы, топики, действия, сервисы и дерево преобразований систем координат.
- Gazebo Classic. Физически-реалистичный симулятор рабочей среды, поддерживающий плагины омни-движения (`gazebo_ros_planar_move`) и модели сенсоров.
- Navigation 2 (Nav2). Навигационный стек ROS 2, используемый для серверов глобального планировщика, локального контроллера, поведений и BT Navigator.
- `slam_toolbox`. Пакет online-SLAM на базе 2D-лидара.
- OpenCV и `cv_bridge` — для обработки изображений и цветовой сегментации.
- Docker, Docker Compose, `tmux`, `tmuxp` — для контейнеризации среды исполнения и оркестрации стартовой последовательности.
- `Xvfb`, `x11vnc`, `noVNC` — для удалённого доступа к графическим интерфейсам симулятора и RViz независимо от графической подсистемы хоста.

4.2 Состав узлов и их назначение

Ключевые программные узлы и их функции приведены в таблице 4.1.

Таблица 4.1 – Ключевые узлы программного комплекса

Узел	Функция
<code>obstacles_controller</code>	Создание цели и цилиндрических препятствий через сервис <code>/spawn_entity</code> , периодическая публикация случайных скоростей в топики <code>/<name>/cmd_vel</code> для имитации динамической среды.
<code>target_detector</code>	Детекция синего объекта в кадре камеры в пространстве HSV, оценка дальности до цели по данным 2D-лидара, публикация флага видимости <code>/target_visible</code> и позы <code>/target_pose</code> в системе координат карты.
<code>target_nav_bridge</code>	Формирование и отправка целей <code>NavigateToPose</code> в Nav2 по данным детектора, запуск поведения <code>Spin</code> при потере цели для её поиска.
<code>scan_bridge</code>	Ретрансляция топика <code>/omni_robot/scan</code> в <code>/scan</code> для совместимости с настройками SLAM и costmap.
<code>cmd_vel_bridge</code>	Ретрансляция управляющих скоростей Nav2 из <code>/cmd_vel</code> в топик плагина движения робота <code>/omni_robot/cmd_vel</code> .
<code>odom_tf_bridge</code>	Публикация преобразования <code>odom</code> $\rightarrow$ <code>base_link</code> по данным одометрии плагина движения.
<code>controller_server</code> (Nav2)	Загрузка плагина <code>MPPIController</code> с настроенным набором критиков, вычисление команд скорости на каждом такте.
<code>slam_toolbox</code>	Построение карты помещения в режиме online, публикация карты и преобразования <code>map</code> $\rightarrow$ <code>odom</code> .

4.3 Модель робота

Модель робота описана в формате URDF/хасго и включает:

- базовое тело `base_link` в виде параллелепипеда с задаваемыми габаритами для аппроксимации корпуса робота (по умолчанию $0,3 \times 0,3 \times 0,1$ м);
- 2D-лидар на верхней грани корпуса с углом обзора 360° , 360 лучами и диапазоном 0,1–10 м (плагин `gazebo_ros_ray_sensor`);
- фронтальную RGB-камеру с разрешением 640×480 и горизонтальным

полем зрения 1,047 рад (плагин `gazebo_ros_camera`);

- плагин движения `libgazebo_ros_planar_move.so`, принимающий на вход сообщение `geometry_msgs/Twist` с компонентами v_x , v_y и ω , что соответствует модели (3.4).

Параметры модели вынесены в описание робота и меняются без изменения логики контроллера, что соответствует принципу параметризации, сформулированному в требованиях.

4.4 Среда симуляции и подвижные сущности

В качестве базовой среды симуляции используется мир квартиры `apartment.world`, построенный на основе открытой модели AWS RoboMaker (`small_house`). Мир содержит стены, мебель и свободное пространство, достаточное для маневрирования робота.

Подвижные сущности создаются пакетом `apartment_sim` в виде цилиндрических моделей с плагином движения `libgazebo_ros_planar_move.so`. Узел `obstacles_controller` публикует в их топики `cmd_vel` случайные скорости, формируя сценарий с движением цели и препятствий в ограниченной области. Параметры (число препятствий, скорости, места появления, геометрия цилиндров) задаются через ROS-параметры.



Рисунок 4.1 – Сцена симуляции квартиры



Рисунок 4.2 – Сцена симуляции квартиры (вид сверху)

4.5 Контур восприятия цели

Узел `target_detector` реализован на Python с использованием OpenCV и `cv_bridge`. На каждом кадре камеры выполняются следующие шаги:

1. преобразование кадра из пространства BGR в HSV;
2. бинаризация маски по диапазону оттенка синего (`hsv_blue_lower`,

`hsv_blue_upper`) с дополнительными критериями доминирования синего канала над красным и зелёным;

3. морфологическое открытие с заданным размером ядра;
4. выделение связных компонент и выбор наибольшего контура;
5. оценка горизонтального углового направления на цель по нормированной координате центра цели и горизонтальному полю зрения камеры;
6. оценка дальности до цели по данным лидара в окне индексов вокруг полученного направления (медиана допустимых значений);
7. пересчёт полученной точки в координатную систему `map` через TF.

Опубликованные топики:

- `/target_visible` (`std_msgs/Bool`) — флаг видимости цели;
- `/target_pose` (`geometry_msgs/PoseStamped`) — поза цели в карте;
- `/target_marker` (`visualization_msgs/Marker`) — маркер для визуализации в RViz;
- `/omni_robot/camera/image_raw/target_status` — аннотированный кадр с индикатором видимости.

Такой набор выходов позволяет модулю MPPI и верхнему уровню логики использовать только высокоуровневые признаки цели, без привязки к деталям обработки изображения.

4.6 Формирование навигационных целей

Узел `target_nav_bridge` связывает контур восприятия с действием `Nav2 NavigateToPose`. Его логика:

1. при наличии актуальной позы цели и отличии её от последней отправленной более чем на заданный порог — обновить цель `Nav2`;
2. при переходе `/target_visible` из `True` в `False` — сначала позволить контуру довести движение до последней известной позы, затем

- запустить режим **Spin** (вращение на месте) для поиска цели;
3. при возврате видимости — отменить режим **Spin** и вернуться в режим сопровождения.

Такой конечный автомат уменьшает число резких переключений цели и стабилизирует поведение при временной потере наблюдаемости.

4.7 Локальное управление с MPPI

Локальным контроллером в стеке Nav2 служит плагин `nav2_mppi_controller::MPPIController`, работающий в режиме `motion_model = Omni`, что соответствует модели (3.4). Основные параметры конфигурации (файл `config/omni_nav2_params.yaml`):

- горизонт прогнозирования: $N = 56$ шагов;
- шаг дискретизации: $\Delta t = 0,05$ с;
- число сэмплов: $M = 2000$;
- стандартные отклонения возмущений по v_x, v_y, ω : 0,24, 0,24, 0,8 соответственно;
- верхние/нижние границы скоростей: $v_x \in [-0,65; 0,95]$ м/с, $v_y \in [-0,95; 0,95]$ м/с, $\omega \in [-3,8; 3,8]$ рад/с;
- температура $\lambda = 0,3$;
- частота работы контроллера 50 Гц.

Применён следующий набор критиков, соответствующий слагаемым функции (3.24):

- **ConstraintCritic** — штраф за нарушение ограничений скорости (3.11);
- **CostCritic**, **ObstaclesCritic** — изотропный штраф за приближение к препятствиям на основе costmap, реализация $\Psi_{\text{obs},k}$ (3.25);
- **PotentialFieldCritic** — направленный штраф $\Psi_{\text{pf},k}$ (3.31), реализованный в рамках ВКР;
- **GoalCritic** — штраф за отклонение конечной позы от цели

(терминальное слагаемое);

- **TargetCameraCritic** — пользовательский критик центрирования цели в поле зрения камеры, реализующий слагаемое $w_\phi (e_k^\phi)^2$ из (3.24);
- **PathAlignCritic**, **PathFollowCritic** — штрафы, удерживающие траекторию рядом с путём глобального планировщика;
- **TwirlingCritic** — штраф за избыточное вращение.

Критики **GoalAngleCritic** и **PathAngleCritic** в данной конфигурации отключены: для омни-платформы желаемая ориентация в финальной позе и угол сближения с путём не существенны, а центрирование цели обеспечивается **TargetCameraCritic**.

4.7.1 Пользовательский критик **TargetCameraCritic**

В методе **score** критика:

1. получается последняя опубликованная поза цели `/target_pose` и проверяется её свежесть по таймауту;
2. поза цели пересчитывается в кадр траекторий контроллера (глобальная карта) через TF;
3. для каждой (batch, timestep) пары вычисляется желаемый азимут на цель $\phi_{k+j}^{*(m)} = \text{atan2}(y^t - y_{k+j}^{(m)}, x^t - x_{k+j}^{(m)})$;
4. формируется средняя по траектории абсолютная ошибка $|e_k^\phi|$ между желаемым азимутом и углом рыскания корпуса;
5. получившееся значение, умноженное на настроечный вес, в степени **cost_power** добавляется в суммарную стоимость траекторий MPPF.

Критик автоматически отключается при устаревании топики `/target_pose`, что согласовано с логикой узла **target_nav_bridge** и поддерживает корректное поведение при потере цели.

4.7.2 Пользовательский критик `PotentialFieldCritic`

Критик реализует направленный штраф $\Psi_{\text{pf},k}$ (3.31), восполняющий ограничение изотропных штрафов `CostCritic` и `ObstaclesCritic` для голономных движений. На каждом такте для текущей позы робота и для точек сэмплированных траекторий выполняется:

1. чтение значений `costmap` в точке и вокруг неё (радиус `activation_sample_radius`), если максимум ниже порога активации `activation_cost_threshold` — критик возвращает нулевую стоимость;
2. вычисление расстояния до препятствия $\tilde{d}(c)$ по обратной характеристике инфляционного слоя `costmap` (параметры `inflation_radius`, `cost_scaling_factor` — те же, что у `InflationLayer`);
3. оценка двухточечного градиента ∇c по двум разностям `gradient_sample_distance` (с весами 0,7 и 0,3) для устойчивости к численному шуму `costmap` и построение направления отталкивания $\hat{\mathbf{f}}$ (3.28);
4. суммирование вдоль траектории до прохождения дистанции `influence_distance`: проксимальная составляющая $w_\rho \rho_k$ и направленная составляющая $w_\sigma (1 + \rho_k) \sigma_k$ из (3.29), (3.30);
5. ограничение полученной стоимости снизу нулём и сверху параметром `max_force_cost` с последующим возведением в степень `cost_power` и суммированием с весом `cost_weight`, помноженным на масштаб `dominance_scale`.

Внутри радиуса `near_goal_distance` от текущей цели Nav2 критик выключается, чтобы не препятствовать сближению с финальной позой. Для отладки в топик `/potential_field_critic/markers` публикуется визуализация направления $\hat{\mathbf{f}}$ из текущей позы робота, что позволяет проверять корректность оценки градиента в RViz.

4.8 SLAM

Для построения карты и локализации используется `slam_toolbox` в асинхронном режиме. Параметры (файл `config/slam.yaml`): решатель Ceres с линейным методом `SPARSE_NORMAL_CHOLESKY` и стратегией Levenberg–Marquardt, разрешение карты 0,05 м, максимальная дальность лидара 15 м, интервал обновления карты 1 с.

Навигационный стек использует `map` как глобальный фрейм (для планировщика и глобальной `costmap`) и `odom` как локальный фрейм (для локальной `costmap` и локального контроллера), что соответствует рекомендациям Nav2 по разделению ответственности.

4.9 Пример рабочей сцены

Пример рабочего состояния системы в процессе сопровождения цели приведён на рисунке 4.3. На снимке экрана RViz видно:

- красная стрелка - направление действия `PotentialFieldCritic`,
- зеленая стрелка - ориентация робота,
- оранжевая стрелка - направление на цель,
- построенную карту помещения,
- глобальную `costmap`,
- показания 2D-лидара,
- глобальный путь и маркер цели,
- кадр фронтальной камеры с наложенным индикатором видимости цели.

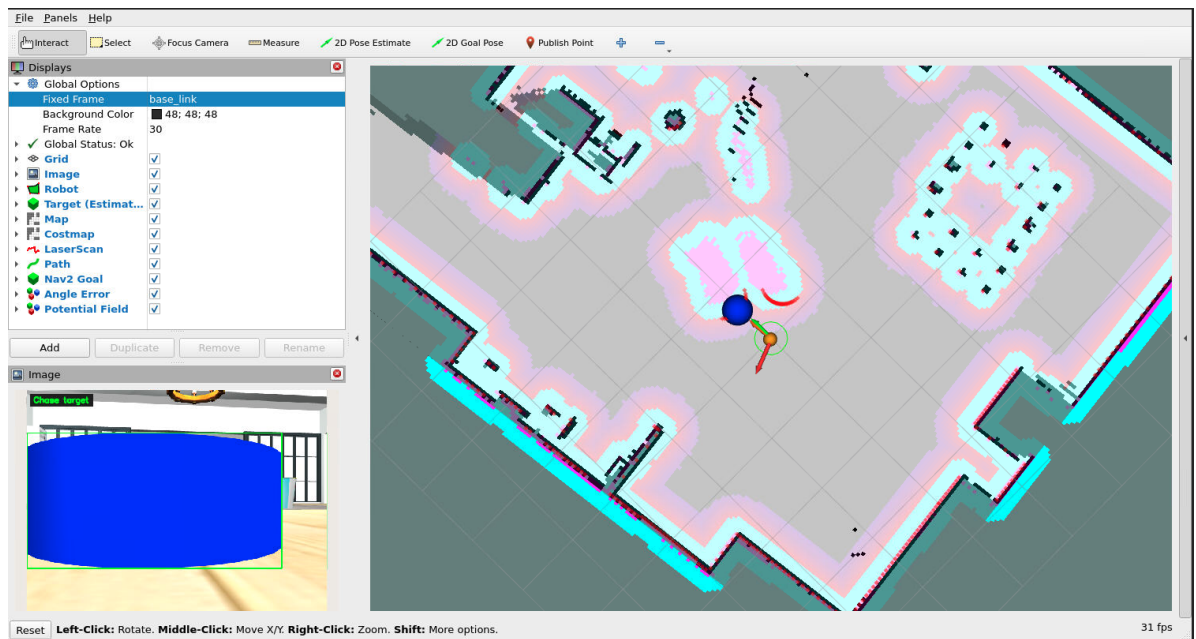


Рисунок 4.3 – Рабочая сцена в RViz: карта помещения, costmap, лидар, путь и маркер цели

5 ЭКСПЕРИМЕНТАЛЬНАЯ ЧАСТЬ

В настоящем разделе описана методика количественной оценки разработанного решения, состав метрик, состав серии экспериментов и полученные результаты.

5.1 Методика проведения серии запусков

Все запуски проводятся в одинаковой сцене и с одинаковой конфигурацией Nav2/MPPI, различие между запусками определяется случайными векторами скоростей подвижных препятствий и стохастикой сэмплирования MPPI.

5.2 Метрики

5.2.1 Распределение режимов сопровождения

По данным топика `/target_nav_mode` строится временная диаграмма режимов и интегральная доля времени каждого режима за запуск:

$$\bar{\tau}_m = \frac{1}{T} \int_0^T \mathbf{1}[m(t) = m] dt, \quad m \in \{\text{Сопровождение, К последней позиции, Поиск}\}, \quad (5.1)$$

где T — длительность запуска. Высокая доля режима Сопровождение цели означает, что робот в большинстве времени двигался непосредственно за пользователем, а не выполнял поиск или перемещение к последней известной позиции.

5.2.2 Видимость цели

По бинарному сигналу $v(t) \in \{0,1\}$ из темы `/target_visible` вычисляется относительное время наблюдаемости цели:

$$\bar{v} = \frac{1}{T} \int_0^T v(t) dt. \quad (5.2)$$

5.2.3 Метрики безопасности

По данным топики `/scan` в каждый момент времени вычисляется минимальное расстояние до препятствия $d_{\min}^{\text{lid}}(t) = \min_i r_i(t)$. Введём два пороговых расстояния: d_{col} — величина, ниже которой считается произошедшим контактом с препятствием, и $d_{\text{pot}} > d_{\text{col}}$ — ширина зоны риска. По последовательности значений $d_{\min}^{\text{lid}}(t)$ методом подсчёта переходов с гистерезисом `event_release_seconds` (исключение повторного срабатывания на одном и том же приближении) определяются:

- N_{c} — число эпизодов с $d_{\min}^{\text{lid}} \leq d_{\text{col}}$ (фактические контакты);
- N_{r} — число эпизодов входа в зону риска ($d_{\min}^{\text{lid}} \leq d_{\text{pot}}$);
- $N_{\text{p}} = \max(0, N_{\text{r}} - N_{\text{c}})$ — число случаев, когда система зашла в зону риска, но избежала контакта;
- $\rho = N_{\text{p}}/N_{\text{r}}$ ($\rho := 1$ при $N_{\text{r}} = 0$) — доля предотвращённых контактов.

Подсчёт ведётся начиная с первого момента, когда лидар фиксирует $d_{\min}^{\text{lid}} > d_{\text{pot}}$, чтобы исключить артефакты этапа спавна робота. В работе принято $d_{\text{col}} = 0,30$ м, $d_{\text{pot}} = 0,50$ м, `event_release_seconds` = 0,5 с.

5.2.4 Плавность управления

По данным топики `/cmd_vel` ($\mathbf{u}_k = [v_{x,k}, v_{y,k}, \boldsymbol{\omega}_k]^\top$) вычисляется норма приращения управления и её среднеквадратичное значение по времени

запуска:

$$\text{RMS}(\|\Delta \mathbf{u}\|_2) = \sqrt{\frac{1}{K-1} \sum_{k=1}^{K-1} \|\mathbf{u}_k - \mathbf{u}_{k-1}\|_2^2}, \quad (5.3)$$

где K — число опубликованных управлений. Меньшие значения соответствуют более плавному поведению контура управления и непосредственно интерпретируются как минимизация слагаемого $w_{\Delta u} \|\mathbf{u}_k - \mathbf{u}_{k-1}\|_2^2$ в (3.24).

5.3 Результаты одного запуска

В качестве иллюстрации поведения системы на рисунках 5.1–5.4 приведены графики одного из запусков серии экспериментов.

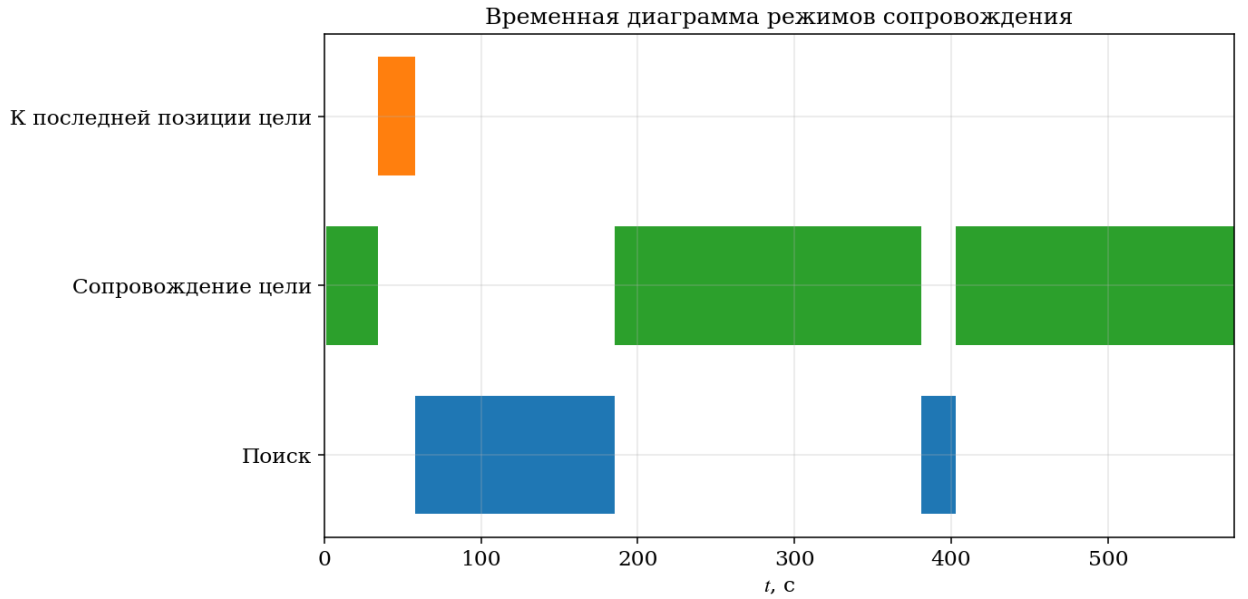


Рисунок 5.1 – Временная диаграмма режимов сопровождения для отдельного запуска

На временной диаграмме режимов (рисунок 5.1) видна типичная картина: робот в основном находится в режиме Сопровождение цели (зелёные интервалы), кратковременно переходя в режим Поиск, когда цель временно исчезает из кадра, после чего возобновляет сопровождение. Режим К последней позиции цели активируется эпизодически и кратко, что

соответствует логике узла `target_nav_bridge`.

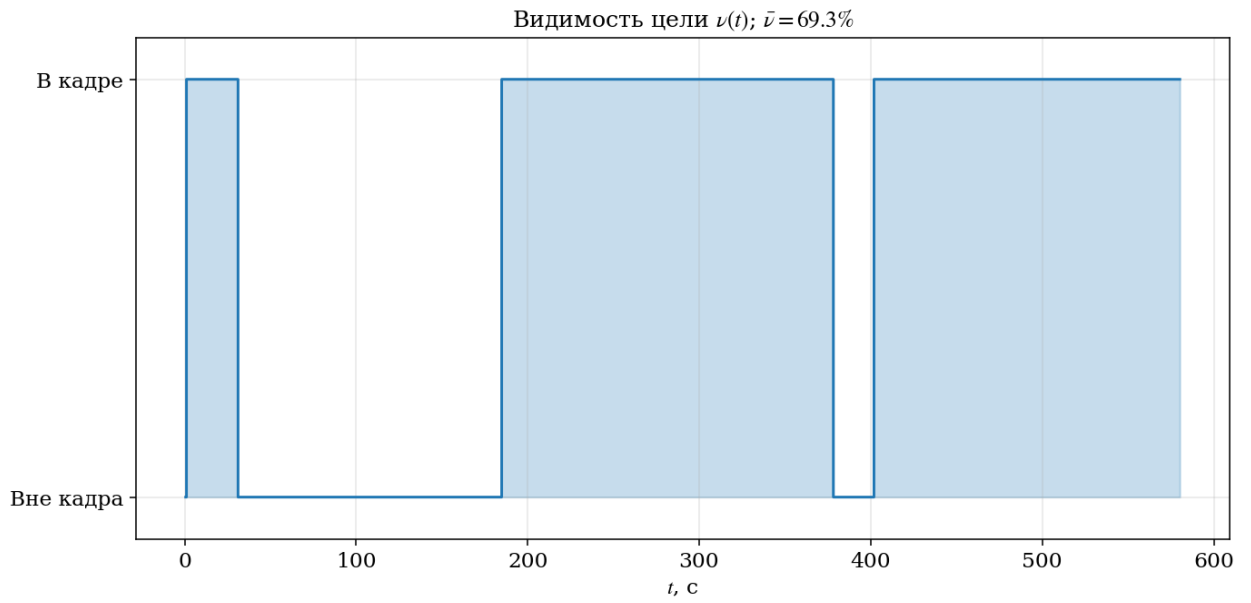


Рисунок 5.2 – Видимость цели $v(t)$ для отдельного запуска

График видимости (рисунок 5.2) показывает суммарную долю времени с целью в кадре $\bar{v} \approx 69\%$, падения до нуля соответствуют эпизодам, когда подвижные препятствия закрывали цель, и совпадают с переключениями в режимы поиска на рисунке 5.1.

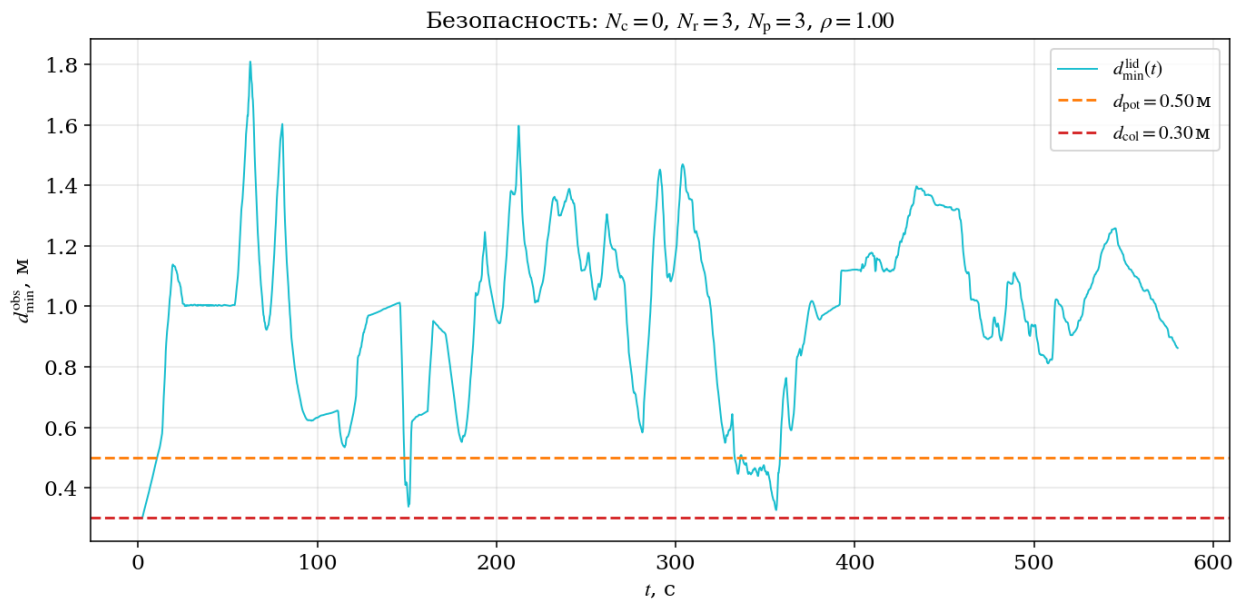


Рисунок 5.3 – Минимальное расстояние до препятствия по данным лидара с горизонтальными порогом d_{pot} и d_{col}

На графике безопасности (рисунок 5.3) видны зоны кратковременного захода ниже d_{pot} , при этом для данного запуска фактических контактов не зафиксировано ($N_c = 0$, $N_r = 3$, $N_p = 3$, $\rho = 1,00$). Это иллюстрирует совместный эффект штрафов $\Psi_{\text{obs},k}$ (3.25) и $\Psi_{\text{pf},k}$ (3.31): робот допускает заход в зону риска, но не доводит ситуацию до контакта.

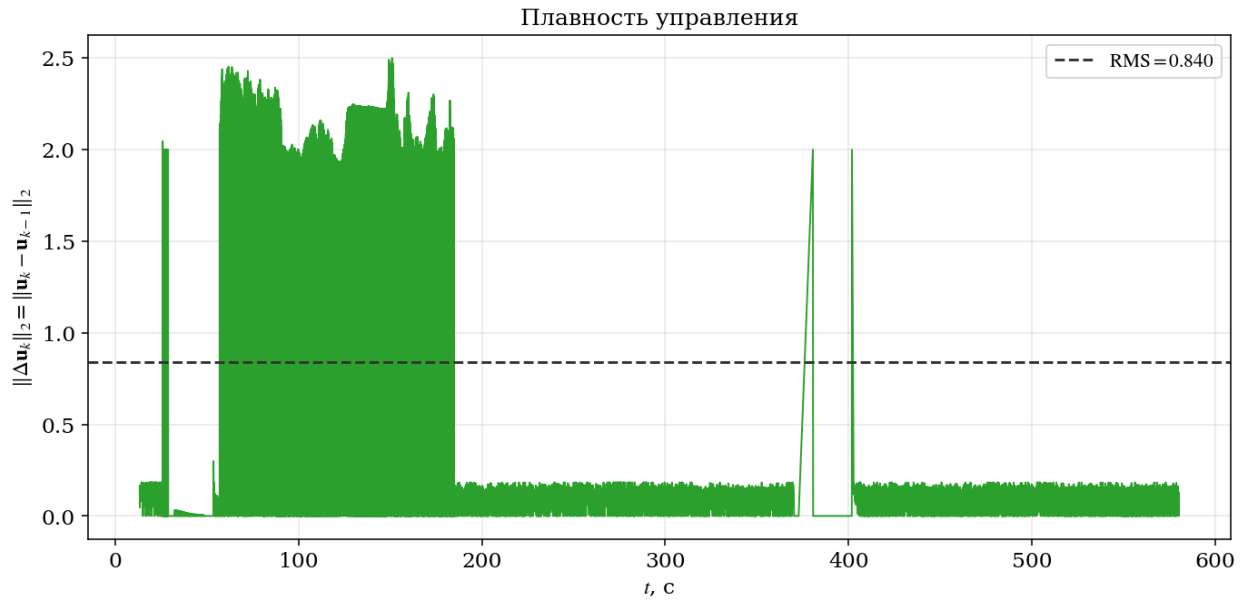


Рисунок 5.4 – Норма приращения управления $\|\Delta \mathbf{u}_k\|_2$ для отдельного запуска

На графике плавности управления (рисунок 5.4) повышенные значения в отдельных интервалах соответствуют моментам маневрирования при обходе препятствий и переключений режимов, длительные участки сопровождения дают близкие к нулю приращения управления.

5.4 Агрегированные результаты по серии экспериментов

На рисунках 5.5–5.8 приведены агрегированные характеристики по всем 40 запускам.

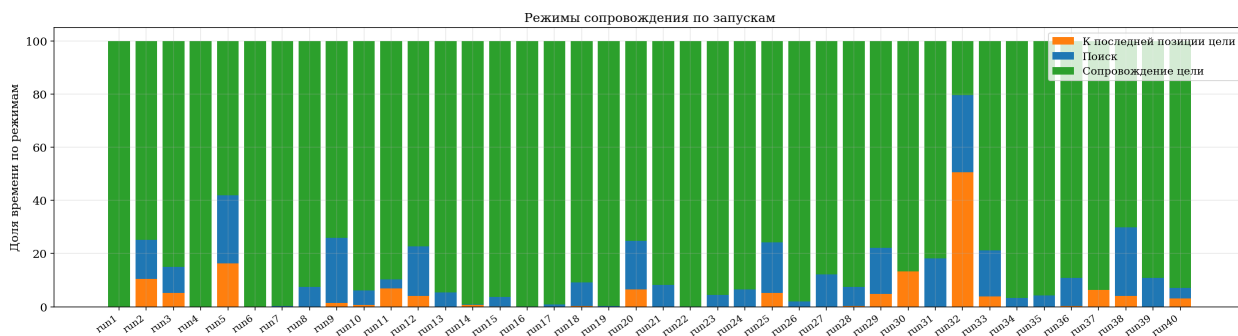


Рисунок 5.5 – Доли времени по режимам сопровождения по запускам

Распределение режимов (рисунок 5.5) показывает, что доминирующим режимом во всех запусках является Сопровождение цели, режимы Поиск и К последней позиции цели активируются только при кратковременных потерях наблюдаемости и в большинстве запусков занимают совокупно менее 25% времени, что подтверждает корректность логики переключения и устойчивость трекера.

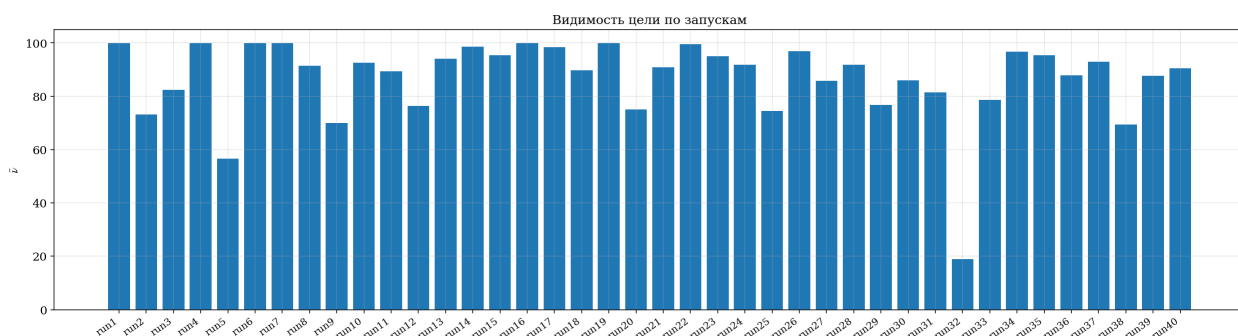


Рисунок 5.6 – Доля времени видимости цели $\bar{V}$ по запускам

Видимость цели (рисунок 5.6) в большинстве запусков превышает 80%, характерные просадки до 60–70% соответствуют сценариям, в которых подвижные препятствия систематически закрывали цель, а единичная просадка ниже 20% (run32) — случаю, когда цель оказалась за углом помещения и удерживалась там подвижным препятствием в течение значительной части запуска.

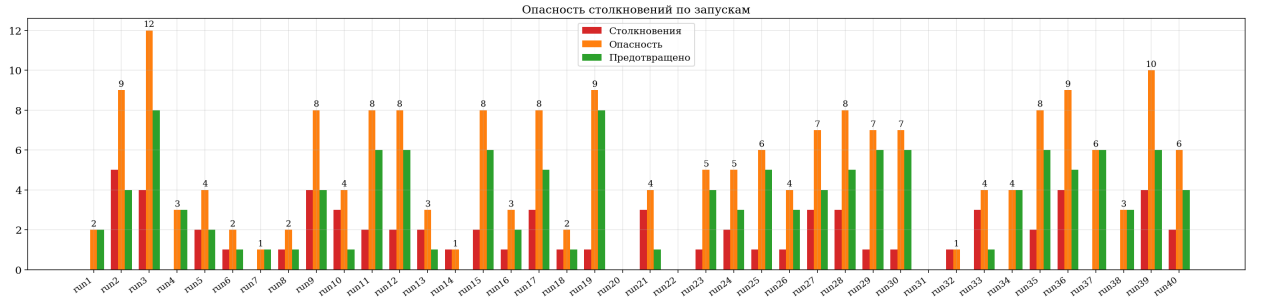


Рисунок 5.7 – Безопасность: число столкновений N_c , заходов в зону риска N_r и предотвращённых контактов N_p по запускам

Агрегированная характеристика безопасности (рисунок 5.7) суммирует поведение системы по всем 40 запускам. Видно, что:

- число заходов в зону риска N_r значительно превышает число фактических контактов N_c практически в каждом запуске; это и есть прямой количественный эффект добавления направленного штрафа $\Psi_{pf,k}$ (3.31);
- суммарная по серии доля предотвращённых контактов составляет $\bar{\rho} \approx 0,70$.



Рисунок 5.8 – Среднеквадратичная норма приращения управления $RMS(||\Delta \mathbf{u}_k||_2)$ по запускам

Среднеквадратичная норма приращения управления (рисунок 5.8) распределена в диапазоне 0,1–1,1, запуски с повышенными значениями соответствуют сценариям с большим числом маневрирования.

5.5 Обсуждение результатов

Результаты экспериментальной серии подтверждают:

1. функциональную корректность контура восприятия и переключения режимов: средняя доля видимости цели превышает 80%, а доминирующий режим — Сопровождение цели;
2. полезность направленного штрафа $\Psi_{rf,k}$: число фактических контактов N_c систематически меньше числа заходов в зону риска N_r , что не обеспечивается одной лишь изотропной составляющей $\Psi_{obs,k}$ при использовании голономной омни-платформы;
3. приемлемую плавность управления: **RMS** приращений управления остаётся в пределах, обусловленных параметрами сэмплирования МР-PI и ограничениями на ускорения.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы поставленная цель достигнута: разработано программное обеспечение автономного мобильного омни-робота, обеспечивающее сопровождение пользователя в квартирной среде с динамическими препятствиями на базе стохастической реализации модельного прогнозирующего управления.

В рамках выполнения задач ВКР получены следующие основные результаты.

1. Проведён анализ существующих подходов к локальному планированию траектории и методов SLAM, подтверждена целесообразность выбора MPPI как основного локального контроллера и slam_toolbox как решения задачи SLAM на базе 2D-лидара; в качестве интегрирующего программного стека обоснованно выбран Nav2.
2. Построена математическая модель омни-платформы с колёсами Mecanum: в обозначениях кинематической схемы введены связные L , l , ψ , r , ϕ_i , выведены кинематические соотношения в непрерывной и дискретной формах, записаны формулы связи обобщённых скоростей корпуса и угловых скоростей колёс, зафиксированы ограничения управления и условия безопасности относительно препятствий.
3. Формализован принцип модельного прогнозирующего управления и его стохастической реализации MPPI; сформирована составная функция потерь, отражающая одновременные требования по удержанию дистанции до цели, центрированию цели в поле зрения камеры, безопасному обходу препятствий, плавности управления и остановке при потере цели; показана эквивалентность построенной функции потерь совокупности критиков MPPI.
4. Разработана воспроизводимая контейнерная среда исполнения на базе

Docker, ROS 2 Humble и Gazebo, с автоматизированной оркестрацией стартовой последовательности через tmux и удалённым доступом к графическому интерфейсу через noVNC.

5. Реализована модель омни-робота с 2D-лидаром и фронтальной RGB-камерой в формате URDF/хасро; развёрнут набор прикладных ROS-узлов (мосты тем и преобразований, детектор цели, мост действий навигации, контроллер подвижных сущностей).
6. Реализован пользовательский критик `TargetCameraCritic` в составе `nav2_mppi_controller`, непосредственно реализующий слагаемое функции потерь, отвечающее за центрирование цели в поле зрения камеры; критик корректно отключается при потере наблюдаемости цели, что соответствует теоретической формулировке.
7. Реализован пользовательский критик `PotentialFieldCritic`, формирующий направленный (анизотропный) штраф вдоль градиента поля препятствий (слагаемое $\Psi_{pf,k}$ функции потерь) с активацией по порогу `costmap` и автоматическим выключением вблизи финальной позы; критик восполняет ограничение изотропных штрафов `ObstaclesCritic/CostCritic` для голономной омни-платформы.

Таким образом, поставленная задача разработки программно-алгоритмического комплекса для омни-робота в задаче сопровождения пользователя в квартирной среде решена в объёме, достаточном для демонстрации всех требуемых функциональных свойств системы. Получена устойчивая и модульная архитектура, пригодная для дальнейшего расширения: замены цветового детектора на более общий алгоритм распознавания человека, перехода от симуляции к реальной платформе, настройки весов функции потерь под конкретные сценарии эксплуатации.

Перспективы дальнейшей разработки темы включают:

- замену модельного детектора цели на детектор на основе современных

моделей машинного обучения (например, YOLO-класса) для работы с произвольным пользователем в бытовой среде;

- проведение прямого А/В-сравнения по тем же метрикам с отключённым `PotentialFieldCritic` для строгой количественной оценки его вклада в безопасность;
- переход от симуляционной валидации к экспериментам на реальной платформе Mesapim-типа с последующим переопределением весов функции потерь по данным полевых испытаний.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Moreno-Caireta, I. Model Predictive Control for a Mecanum-wheeled Robot Navigating among Obstacles / I. Moreno-Caireta, E. Celaya, L. Ros // IFAC-PapersOnLine. — 2021. — Vol. 54, no. 6. — Pp. 119–125.
- [2] Локальное планирование траектории колесного робота в ограниченной среде на основе модельного прогнозирующего управления / М. Алхаддад, К.В. Миронов, С.А. Дергачев et al. // Робототехника и техническая кибернетика. — 2023. — Vol. 11, no. 3. — Pp. 205–214.
- [3] Sreerambatla, S. Comparison of Multiple SLAM Algorithms using ROS / S. Sreerambatla // International Journal of Science and Research (IJSR). — 2021. — Vol. 10, no. 10.
- [4] Nguyen, Q. H. Performance Evaluation of ROS-based SLAM Algorithms for Handheld Indoor Mapping and Tracking Systems / Q. H. Nguyen, P. Johnson, D. Latham // IEEE Sensors Journal. — 2022.

СПИСОК ИЛЛЮСТРАТИВНОГО МАТЕРИАЛА

1. Рисунок 3.1 — Кинематическая схема омни-платформы с колёсами Mecanum.
2. Рисунок 3.2 — Принцип действия колеса Mecanum и формирование результирующей реакции опоры.
3. Рисунок 4.1 — Сцена симуляции квартиры (вид 1).
4. Рисунок 4.2 — Сцена симуляции квартиры (вид 2).
5. Рисунок 4.3 — Рабочая сцена в RViz: карта помещения, costmap, лидар, путь и маркер цели.
6. Рисунок 5.1 — Временная диаграмма режимов сопровождения для отдельного запуска.
7. Рисунок 5.2 — Видимость цели $v(t)$ для отдельного запуска.
8. Рисунок 5.3 — Минимальное расстояние до препятствия по данным лидара с порогами d_{pot} и d_{col} .
9. Рисунок 5.4 — Норма приращения управления для отдельного запуска.
10. Рисунок 5.5 — Доли времени по режимам сопровождения по запускам.
11. Рисунок 5.6 — Доля времени видимости цели по запускам.
12. Рисунок 5.7 — Безопасность: число столкновений, заходов в зону риска и предотвращённых контактов по запускам.
13. Рисунок 5.8 — Среднеквадратичная норма приращения управления по запускам.
14. Таблица 4.1 — Ключевые узлы программного комплекса.